Lab: Connect Identity Server 7.3 to API Manager 4.7 and Keep Existing Client Credentials

What you will build: WSO2 Identity Server 7.3.0 registered as a **third-party Key Manager** in WSO2 API Manager 4.7.0, with a consumer's existing (Fiorano) client ID and secret moved into Identity Server and linked to an API Manager application.

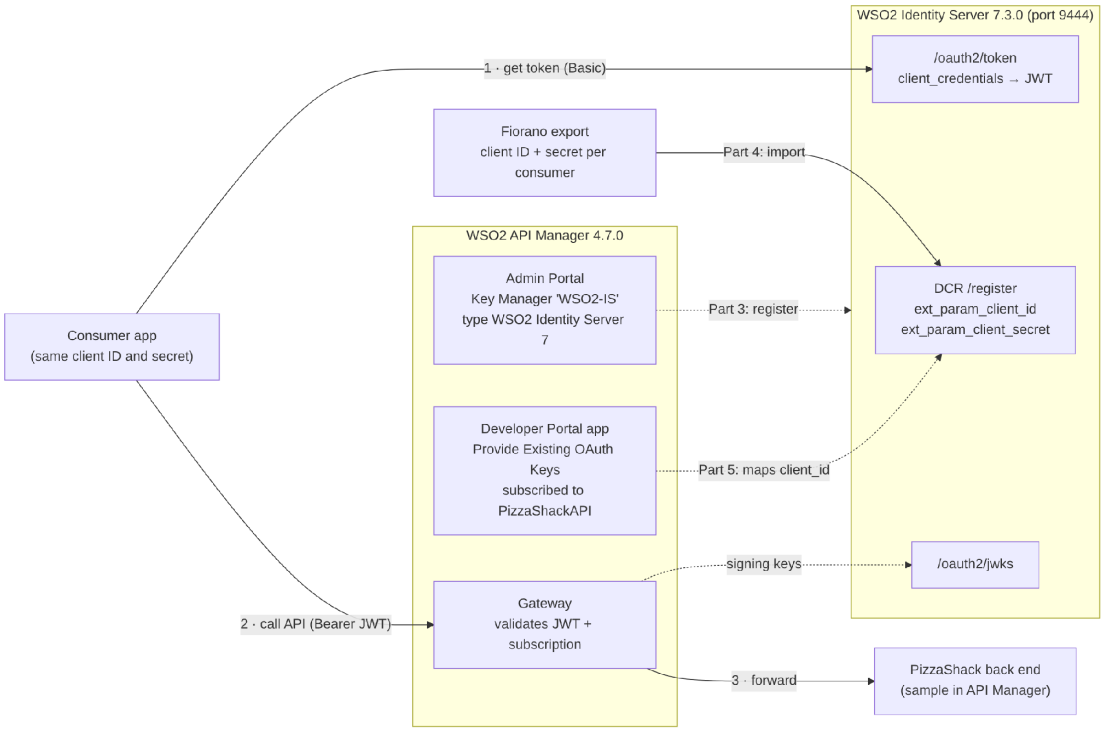
Time: about 75 minutes.

At the end: a consumer uses the **same client ID and secret** it used with Fiorano, gets a JWT from Identity Server, and calls an API on the API Manager gateway.

What you prove	How
API Manager trusts Identity Server's tokens	A JWT from Identity Server is accepted by the gateway
The old credentials still work	Identity Server issues a token for the imported client ID and secret
The credentials belong to an API Manager app	The Developer Portal app shows the same client ID, and its subscription is enforced

This lab is the prerequisite for *Lab: Migrate the Safaricom Hakikisha API from Fiorano to WSO2*, which assumes Identity Server is already connected as a Key Manager.

How it fits together:



Before you start

What you need

- WSO2 API Manager 4.7.0 (`wso2am-4.7.0.zip`), unzipped
- WSO2 Identity Server 7.3.0 (`wso2is-7.3.0.zip`), unzipped
- JDK 21, with `JAVA_HOME` set
- `curl`, `jq` and `keytool` (comes with the JDK)
- About 8 GB of free memory to run both servers on one machine
- No extra back end: the test API is the **PizzaShack** sample that ships with API Manager

Fill in your environment

Write your values in the right-hand column. Every step uses these placeholders.

Placeholder	Meaning	Default	Your value
`<APIM_HOME>`	API Manager install folder	`…/wso2am-4.7.0`	
`<IS_HOME>`	Identity Server install folder	`…/wso2is-7.3.0`	

Placeholder	Meaning	Default	Your value
`<APIM_HOST>`	API Manager host (Admin Portal, Publisher, Dev Portal)	`localhost`	
`<GW_URL>`	Gateway HTTPS URL	`https://<APIM_HOST>:8243`	
`<IS_URL>`	Identity Server URL, **as API Manager reaches it**	`https://localhost:9444`	
`<KM_NAME>`	Name of the Identity Server Key Manager in API Manager	`WSO2-IS`	
`<IS_ADMIN>`	Identity Server admin login	`admin` / `admin`	

API Manager and Identity Server both listen on `9443` by default. On one machine, Identity Server needs a **port offset** of 1, so it listens on `9444`. Part 1 sets this.

Lab credentials

Pretend these are a consumer's existing Fiorano credentials:

	Value
Client ID	`KeyMgrLabClient01`
Client secret	`KeyMgrLabSecret01`

Identity Server only accepts client IDs of letters, digits and _, 15 to 30 characters long (its default rule). Real Fiorano IDs must be checked against this rule early.

Part 1 – Prepare Identity Server (15 min)

Identity Server needs a port offset and a few Key Manager settings.

1. Open `<IS_HOME>/repository/conf/deployment.toml`. In the `[server]` section, add the offset:

```
[server]
hostname = "localhost"
offset = 1
```

2. Add these lines at the end of the file:

```
[oauth]
authorize_all_scopes = true

[[resource.access_control]]
context = "(.*)/scim2/Me"
secure = true
http_method = "GET"
cross_tenant = true
permissions = []
scopes = []
```

What they do: `authorize_all_scopes` lets Identity Server issue the scopes that API Manager defines; the `scim2/Me` rule is used by the UserInfo endpoint.

If `[server]` or `[oauth]` already exists in the file, add the lines to that section instead of creating a second one.

3. Token revocation sync. When a token is revoked in Identity Server, this tells the gateway at once:

   a. Copy the `wso2is.notification.event.handlers` JAR (the 2.x version the API Manager 4.7.0 docs list for Identity Server 7.x) into `<IS_HOME>/repository/components/dropins`.

   b. Add to `deployment.toml`:

```
[[event_listener]]
id = "token_revocation"
type = "org.wso2.carbon.identity.core.handler.AbstractIdentityHandler"
name = "org.wso2.is.notification.ApimOauthEventInterceptor"
order = 1

[event_listener.properties]
notification_endpoint = "https://<APIM_HOST>:9443/internal/data/v1/
notify"
username = "${admin.username}"
password = "${admin.password}"
'header.X-WSO2-KEY-MANAGER' = "WSO2-IS"
```

Check: the file saves without duplicate section names.

Part 2 – Exchange certificates and start the servers (10 min)

API Manager calls Identity Server over HTTPS (to register clients and read signing keys), and Identity Server calls API Manager (revocation). Each must trust the other's certificate.

1. Copy Identity Server's certificate into API Manager's truststore (The default password for the keystores is `wso2carbon`):

```
keytool -export -alias wso2carbon -file is7.crt \
  -keystore <IS_HOME>/repository/resources/security/wso2carbon.p12 \
  -storetype PKCS12 -storepass wso2carbon
keytool -import -noprompt -alias wso2is7 -file is7.crt \
  -keystore <APIM_HOME>/repository/resources/security/client-
truststore.jks \
  -storepass wso2carbon
```

2. Copy API Manager's certificate into Identity Server's truststore:

```
keytool -export -alias wso2carbon -file apim.crt \
  -keystore <APIM_HOME>/repository/resources/security/wso2carbon.jks \
  -storepass wso2carbon
keytool -import -noprompt -alias wso2apim -file apim.crt \
  -keystore <IS_HOME>/repository/resources/security/client-truststore.p12
\
  -storetype PKCS12 -storepass wso2carbon
```

If `keytool` says the certificate already exists under another alias, that is fine: both packs can ship the same default certificate.

3. Start API Manager, wait for `WSO2 Carbon started`, then start Identity Server:

```
<APIM_HOME>/bin/api-manager.sh / <APIM_HOME>/bin/api-manager.bat
<IS_HOME>/bin/wso2server.sh / <IS_HOME>/bin/wso2server.bat
```

4. Check Identity Server from a third terminal:

```
curl -sk <IS_URL>/oauth2/token/.well-known/openid-configuration | jq
-r .issuer
```

Check: it prints `https://localhost:9444/oauth2/token`, and `<IS_URL>/console` opens the Identity Server Console.

Part 3 – Add Identity Server as a Key Manager (15 min)

API Manager learns where Identity Server issues and checks tokens.

1. Open the Admin Portal (`https://<APIM_HOST>:9443/admin`) › Login with the default admin credentials (username: `admin`, password: `admin`) › **Key Managers** › **Add Key Manager**.

2. General details:

Field	Value
Name	`WSO2-IS` (this is `<KM_NAME>`)
Display Name	`WSO2 Identity Server 7.3`
Key Manager Type	**WSO2 Identity Server 7**

3. Key Manager Endpoints: paste the well-known URL and click **Import**:

   ```
   <IS_URL>/oauth2/token/.well-known/openid-configuration
   ```

   Then check every field against this table. Two fields need to be filled in by hand: UserInfo Endpoint (the import fills in /oauth2/userinfo, which does not work here) and Scope Management Endpoint (the import leaves it empty).

Field	Value
Issuer	`<IS_URL>/oauth2/token`

Field	Value
Client Registration Endpoint	`<IS_URL>/api/identity/oauth2/dcr/v1.1/register`
Introspection Endpoint	`<IS_URL>/oauth2/introspect`
Token Endpoint	`<IS_URL>/oauth2/token`
Display Token Endpoint	`<IS_URL>/oauth2/token`
Revoke Endpoint	`<IS_URL>/oauth2/revoke`
Display Revoke Endpoint	`<IS_URL>/oauth2/revoke`
UserInfo Endpoint	`<IS_URL>/scim2/Me`
Authorize Endpoint	`<IS_URL>/oauth2/authorize`
Scope Management Endpoint	`<IS_URL>/api/identity/oauth2/v1.0/scopes`

If the gateway reaches Identity Server by a different host name than your laptop, use the gateway's view for every endpoint except the two *Display* endpoints, which consumers see in the Developer Portal.

4. Grant types: keep what the import selected. **client_credentials** must be ticked.
5. Certificates: choose **JWKS** and enter `<IS_URL>/oauth2/jwks`.
6. Connector Configurations:

Field	Value
Username / Password	`<IS_ADMIN> (username: admin, password: admin)`
WSO2 Identity Server 7 API Resource Management Endpoint	`<IS_URL>/api/server/v1/api-resources`
WSO2 Identity Server 7 Roles Endpoint	`<IS_URL>/scim2/v2/Roles`
Create roles in WSO2 Identity Server 7	Disabled
Key Manager Permission	Public

7. Advanced configurations: make sure Out Of Band Provisioning is ticked (it is ticked by default for this Key Manager type) (existing OAuth apps can be mapped). Keep **Token Generation** and **Oauth App Creation** ticked.

   This is the setting that shows **Provide Existing OAuth Keys** in the Developer Portal. Without it, you can only generate new keys.

8. Keep the rest as default, click **Add**, and make sure the new Key Manager is **enabled** in the list.

   Check: **Key Managers** lists `WSO2-IS`, enabled, next to `Resident Key Manager`.

Part 4 – Put the existing credentials into Identity Server (10 min)

We keep the consumer's client ID and secret. We register them in Identity Server through its DCR endpoint, so Identity Server stores them unchanged instead of generating new ones.

1. Register the client:

```
curl -sk -u admin:admin -H "Content-Type: application/json" \
  -X POST <IS_URL>/api/identity/oauth2/dcr/v1.1/register \
  -d '{
        "client_name": "KeyMgrLabConsumer",
        "grant_types": ["client_credentials"],
        "ext_param_client_id": "KeyMgrLabClient01",
        "ext_param_client_secret": "KeyMgrLabSecret01",
        "token_type_extension": "JWT"
      }' | jq
```

   Check: `client_id` in the reply is `KeyMgrLabClient01` and `client_secret` is `KeyMgrLabSecret01`. In the Identity Server Console › **Applications**, `KeyMgrLabConsumer` is listed.

   `token_type_extension: JWT` makes Identity Server issue self-contained JWTs, which the gateway checks with the JWKS keys. You change nothing in the Console.

2. Get a token straight from Identity Server with the old credentials:

```
curl -sk -u KeyMgrLabClient01:KeyMgrLabSecret01 \
  -d grant_type=client_credentials <IS_URL>/oauth2/token | jq
```

   Check: `200` with an `access_token` that is a JWT: it starts with `eyJ` and has three parts separated by dots.

3. Optional: decode the token's middle part at a JWT viewer you trust, or with `cut -d. -f2 | base64 -d`.

   Check: `iss` is `<IS_URL>/oauth2/token` and `client_id` (or `azp`) is `KeyMgrLabClient01`.

Part 5 – Deploy the PizzaShack API and map the keys (15 min)

Deploy the PizzaShack sample API

1. Open the Publisher (`https://<APIM_HOST>:9443/publisher`) › Login with the default admin credentials (username: `admin`, password: `admin`). On the APIs page, click **REST API** › **Deploy Sample API**.

   If other APIs already exist and the button is not shown, open **Create API** › **REST API** › **Deploy Sample API**.

   Check: `PizzaShackAPI` version `1.0.0` appears, already **Published**, with context `/pizzashack`.

   Check: the Overview page shows the gateway URL `…/pizzashack/1.0.0`, and **Resources** lists `GET /menu`.

Map the existing keys

2. Open the Developer Portal (`https://<APIM_HOST>:9443/devportal`) › Login with the default admin credentials (username: `admin`, password: `admin`) › **Applications** › **Add New Application**: name `KeyMgrLabApp` › **Save**.

3. **Production Keys** › the WSO2 IDENTITY SERVER 7.3 tab (the tab shows the Display Name you gave the Key Manager) › **Provide Existing OAuth Keys**:

    a. Consumer Key: `KeyMgrLabClient01`

    b. Consumer Secret: `KeyMgrLabSecret01`

    c. Click **Provide** (or **Save**).

    Check: the page now shows Consumer Key `KeyMgrLabClient01`, not a new generated value.

4. **Subscriptions › Subscribed APIs › SUBSCRIBE** › subscribe to `PizzaShackAPI 1.0.0` with **Unlimited**.

    Check: the application shows one subscription.

Part 6 – Test (10 min)

1. Get a token from Identity Server with the old credentials and keep it in a variable:

```
TOKEN=$(curl -sk -u KeyMgrLabClient01:KeyMgrLabSecret01 \
  -d grant_type=client_credentials <IS_URL>/oauth2/token | jq
-r .access_token)
```

2. Call the PizzaShack menu through the gateway:

```
curl -sk -H "Authorization: Bearer $TOKEN" <GW_URL>/pizzashack/1.0.0/menu
```

3. Run the other cases in the table. Each one needs a single change to the commands above.

#	Case	You should see
1	Call the API with a valid token	`200` with the pizza menu (a JSON list of pizzas)
2	Token request with the right secret	`200`, JWT access token
3	Token request with a wrong secret	`401`, `invalid_client`
4	Call the API with no `Authorization` header	`401`, code `900902` Missing Credentials
5	Call the API with a broken token (change one letter)	`401`, code `900901` Invalid Credentials

#	Case	You should see
6	Unsubscribe the app, get a new token, call again	`403`, code `900908` Resource forbidden

Done when: cases 1–6 behave as in the table. Subscribe the app again after case 6.